

CFD Project 2

November 10, 2025

Eleni Kambouris

Abstract – This report examines the accuracy of different schemes to numerically solve the linear advection equation. Specifically, Lax-Friedrichs, First-order Upwind, Lax-Wendroff, and Minimum Dispersion. Each of these schemes is used to try and match initial conditions and later, a shock condition. Additionally, an updated flux formula without scheme dependency is introduced. The schemes are tested on a row vector with $M = 32$ grid spacings and computed to a given time T . We calculate L1 errors, phase errors, and amplitudes at 4 different CFL numbers.

Intro – When running a simulation, it is important to set the appropriate parameters. This project explores what happens when those parameters, and the formulas we input them into change. Throughout the whole entire project we have 4 different schemes, applied as special choices of q

$$q(v) = 1$$

$$q(v) = |v|$$

$$q(v) = v^2$$

$$q(v) = \frac{1}{3} + \frac{2}{3}v^2$$

these are Lax-Friedrichs, First-order Upwind, Lax-Wendroff, and Minimum-Dispersion respectively.

These are all being put into our flux formula

$$F_{j+\frac{1}{2}}^{n+\frac{1}{2}} = \frac{1}{2} \left(f(u_j^n) + f(u_{j+1}^n) \right) - \frac{q(v)}{2} \frac{\Delta x}{\Delta t} (u_{j+1}^n - u_j^n)$$

The flux formula flows into the update formula

$$U_j^{n+1} - U_j^n = -\frac{\Delta t}{\Delta x} (F_{j+\frac{1}{2}}^{n+\frac{1}{2}} - F_{j-\frac{1}{2}}^{n+\frac{1}{2}})$$

However, we can skip using the flux formula if we just use the scheme derived into the form

$$u_j^{n+1} = \sum c_k u_{j+k}^n$$

Which works out as seen in Appendix B –

$$u_j^n = c_{k-1} u_{j-1}^n + c_k u_j^n + c_{k+1} u_{j+1}^n$$

with

$$c_{-1} = \frac{v + q(v)}{2}, c_0 = 1 - q(v), c_{+1} = \frac{q(v) - v}{2}$$

For Part 2, we use the initial condition

$$u(x, 0) = \sin(2\pi x)$$

for $0 \leq x \leq 1$ and periodic boundary conditions, $u(0, t) = u(1, t)$. We take a uniform step in space between each grid spacing, with $M = 32$ grid spacings. We run the simulation for CFL numbers $v = 1.0, 0.75, 0.50$, and 0.25 . Additionally, the phase errors and amplitudes are calculated and compared to the theoretical results seen from von Neumann analysis.

The L1 error norm is obtained from the formula

$$\epsilon = \frac{1}{M} \sum_{j=1}^M |u_j^N - u_{exact}(x_j, T)|$$

For Part 3, the initial values now simulate a shock discontinuity with

$$u_j^0 = 1, \quad j \leq M/4$$

$$u_j^0 = 0, \quad j > M/4$$

We use a Dirichlet boundary condition at $x=0$ and a Neumann condition at $x=1$. We still use all 4 CFL numbers but now have $aT = 0.5$ and $aT = 1.0$ to test.

Finally, for Part 4, we change the Flux Formula being input to

$$F_{j+\frac{1}{2}}^{n+\frac{1}{2}} = f(u_j^n) + \frac{1-v}{2} B(\Delta u_{j+\frac{1}{2}}, \Delta u_{j-\frac{1}{2}})$$

where $\Delta u_{j+\frac{1}{2}} = u_{j+1}^n - u_j^n$ and

$$|B(x, y)| = \begin{cases} 0, & x, y \text{ are of opposite sign} \\ \max(|x|, |y|), & 0.5 \leq x/y \leq 2.0 \\ 2 \min(|x|, |y|), & \text{else} \end{cases}$$

Finally, there is extra credit, in which the whole entire project is repeated, except our initial u vector is populated with

$$u(x) = \cos^2 \frac{\pi(x - x_i)}{2d}$$

where we have a small interval around x , created from adding or subtracting d from our initial x value. $x_i = 0.25$, and d is selected so that the wave packet can be seen over a few intervals.

Results and Discussion –

The amplitudes and phase errors for both the discrete and the theoretical in addition to the L1 error can be found in Table 1 below.

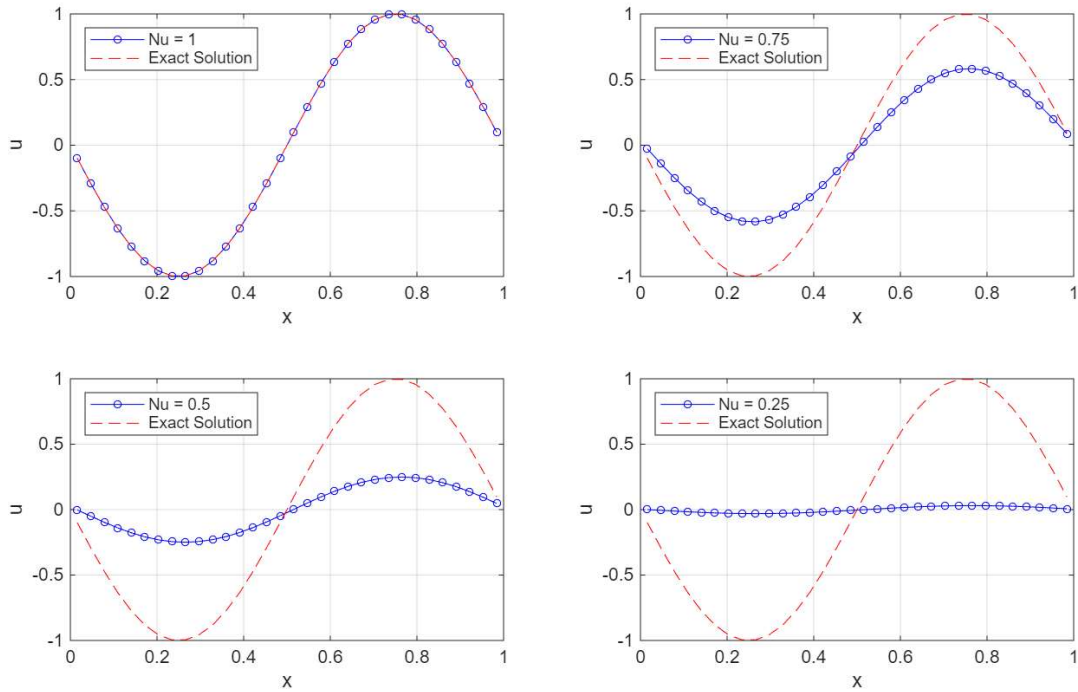
		Discrete		Theoretical		
Scheme	v	Amp Err	Phase Err (deg)	Amp Err	Phase Err	L1
Lax-Friedrichs	1	0	90	0	90	0
Lax-Friedrichs	0.75	-0.4157	86.96	-0.217	88.09	0.2656
Lax-Friedrichs	0.5	-0.751	84.74	-0.387	86.5	0.4795
Lax-Friedrichs	0.25	-0.9694	83.4	-0.579	85.6	0.6183
Upwind	1	0	90	0	90	0
Upwind	0.75	-0.2066	89.57	-0.094	89.73	0.1317
Upwind	0.5	-0.3708	90	-0.127	90	0.2365
Upwind	0.25	-0.5005	91.3	-0.094	90.27	0.3193
Lax-Wendroff	1	0	90	0	90	0
Lax-Wendroff	0.75	-0.0029	91.51	-0.0029	91.02	0.0168
Lax-Wendroff	0.5	-0.0033	92.59	-0.0022	91.18	0.0287
Lax-Wendroff	0.25	-0.0021	93.24	-0.0007	90.74	0.036
Min-Dispersion	1	0	90	0.33	94.2	0
Min-Dispersion	0.75	-0.1652	90	0.14	91.8	0.1053
Min-Dispersion	0.5	-0.3708	90	-0.02	90.6	0.2365
Min-Dispersion	0.25	-0.6856	90	-0.17	90.1	0.4372

We can see that this lines up with our plots below, there is very little to no phase error when our CFL number is equal to 1, and error only increases as we decrease down to 0.25. However, it is interesting that the minimum dispersion is all so close to having very little error, but again it is reflected in the plots as it looks very visually similar.

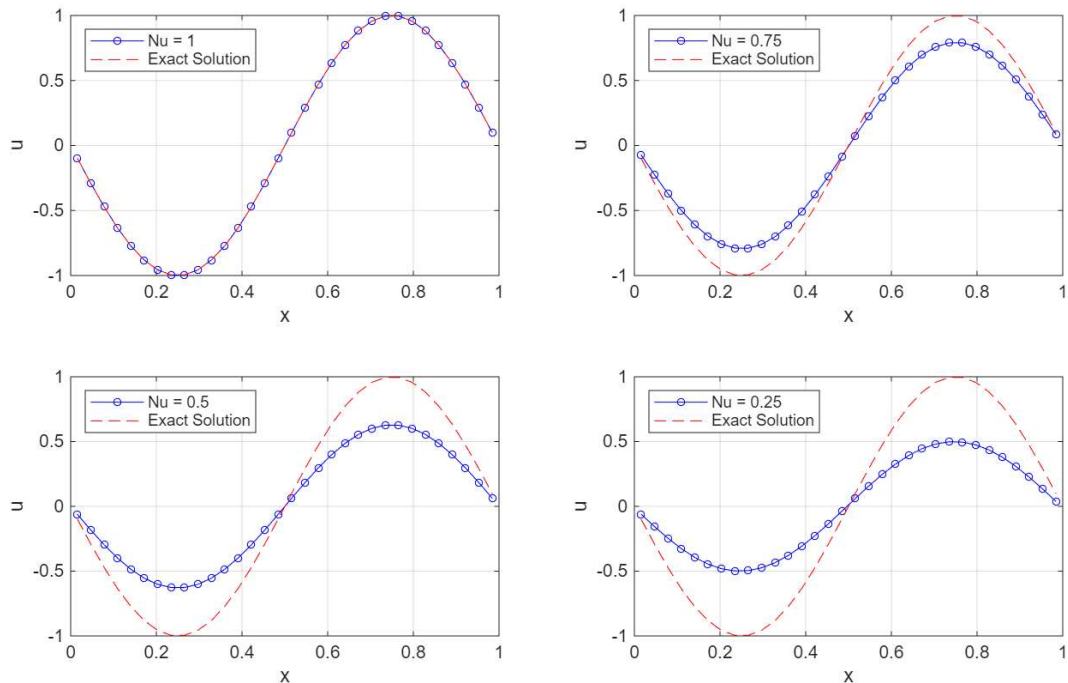
We see the same trend in the L1 error, with CFL = 1 having the least error.

For part 2, we see in all of the figures that $\nu = 1$ provides a pretty exact fit, while $\nu = 0.25$ provides the worst fit. When we compare across schemes we see that Lax-Wendroff converges the quickest, minimum dispersion and upwind are both close seconds, and Lax-Friedrichs sees the most change in results between CFL numbers.

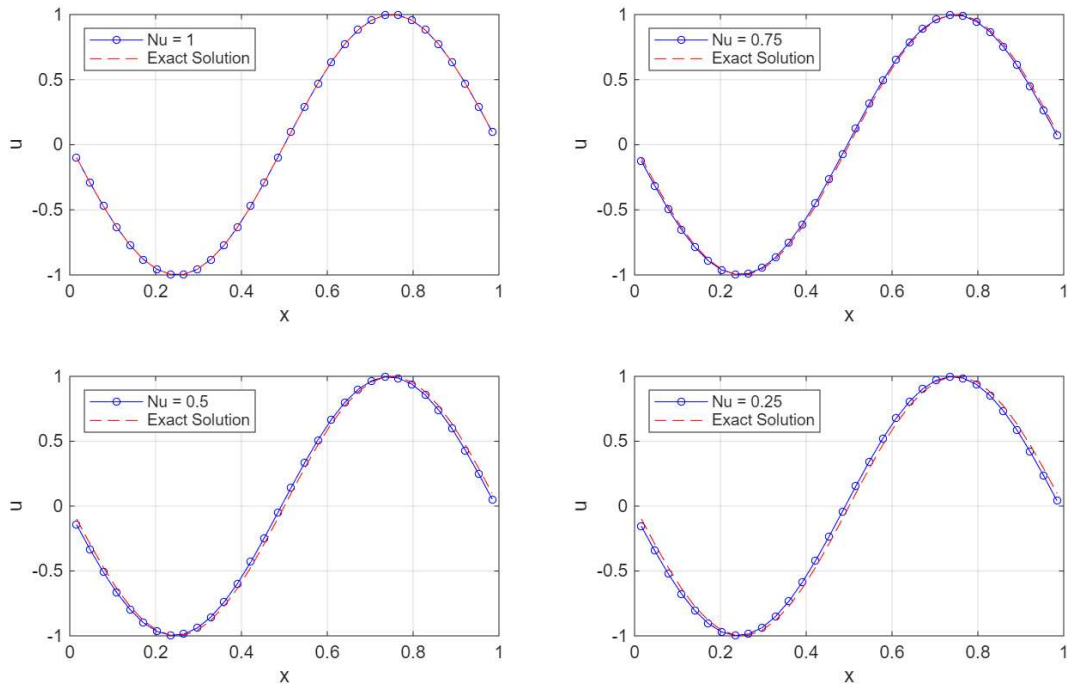
Comparing Nu Values for Lax-Friedrichs



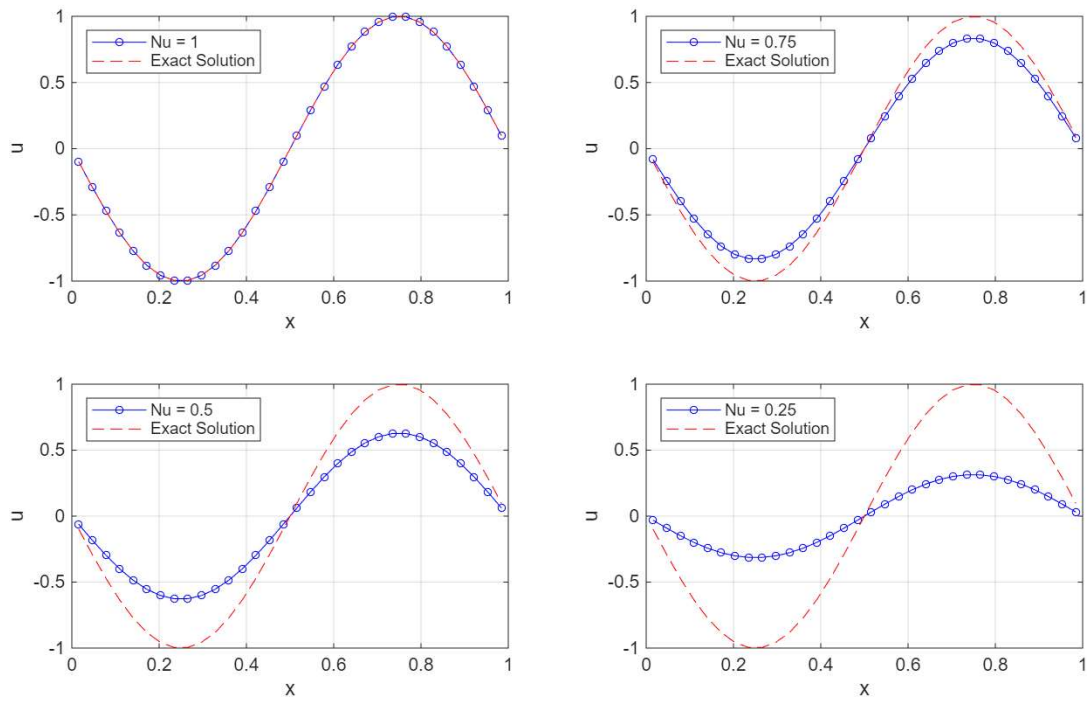
Comparing Nu Values for Upwind



Comparing Nu Values for Lax-Wendroff

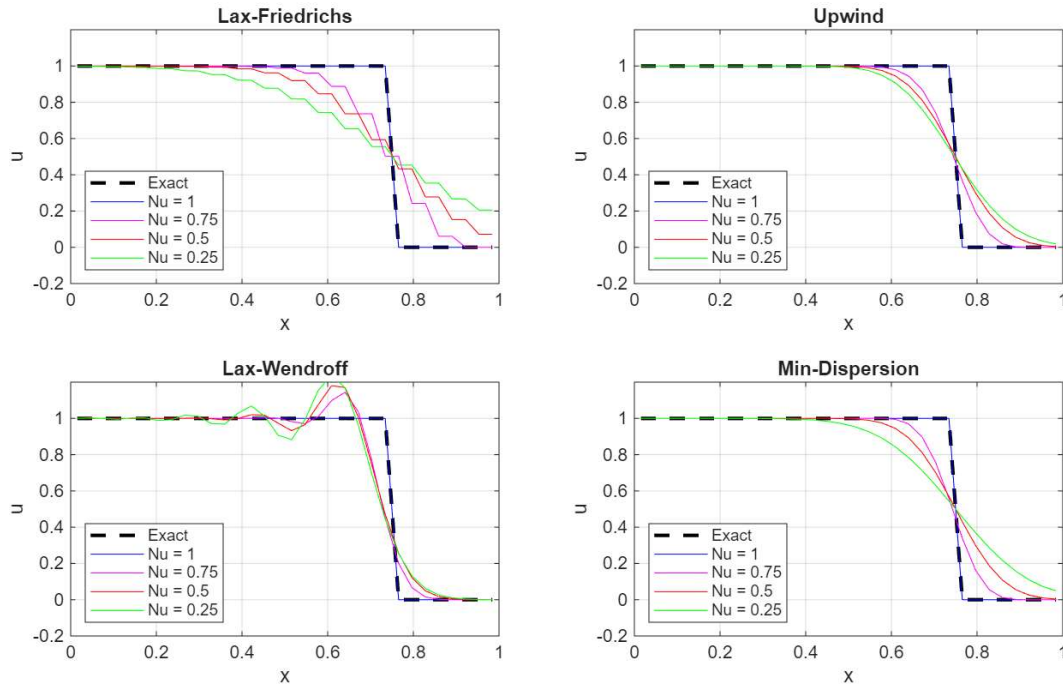


Comparing Nu Values for Min-Dispersion

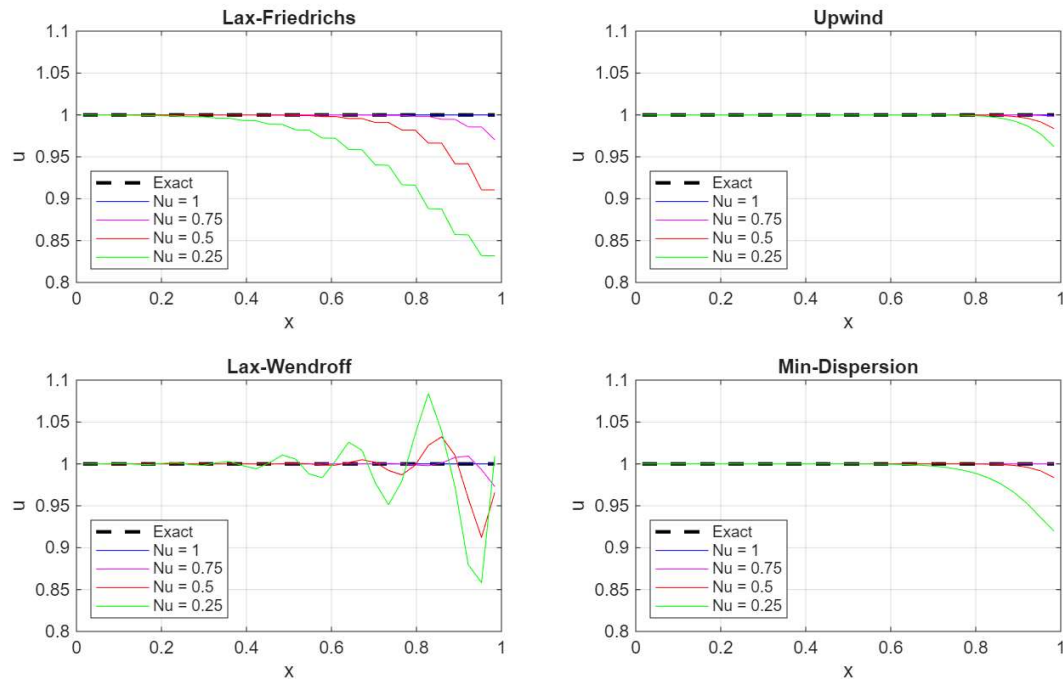


For part 3, we rerun part 2 with different BCs and two aT cases. The results are shown in the figures below.

Comparing Nus for Schemes with aT = 0.5



Comparing Nus for Schemes with aT = 1



In each case, $\nu = 1$ matches the shock behavior exactly, with the trend being increasing accuracy from $\nu = 0.25$. It is also interesting to observe that both Lax-Wendroff and Lax-Friedrichs

exhibit lots of wiggles and oscillations in their solutions, while minimum dispersion and upwind both remain very smooth. However, Lax-Friedrichs simply has oscillations in the path, following a similar behavior to the other two methods, while Lax-Wendroff seems to oscillate back and forth across the shockwave.

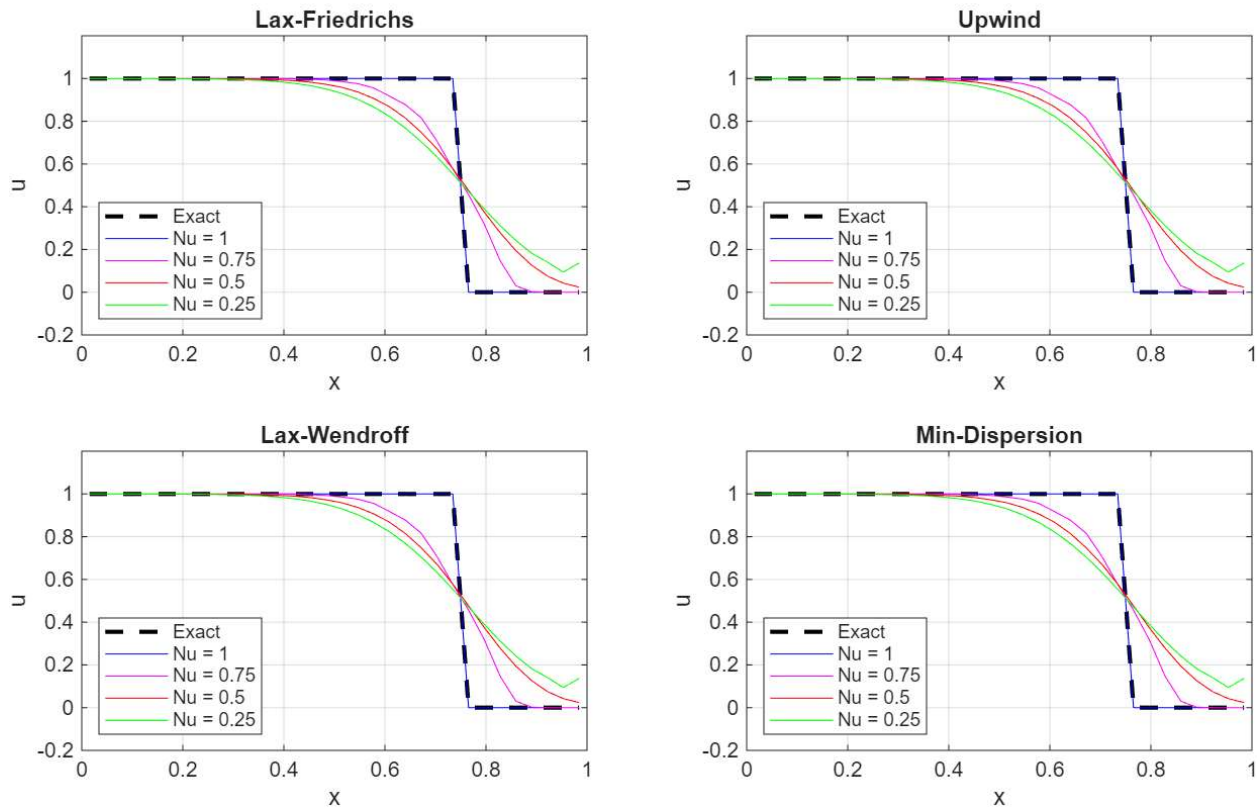
The L1 errors can be seen in the table below.

	nu	L1
Lax-Friedrichs	1	0
Lax-Friedrichs	0.75	0.077
Lax-Friedrichs	0.5	0.1188
Lax-Friedrichs	0.25	0.1755
Upwind	1	0
Upwind	0.75	0.0488
Upwind	0.5	0.0699
Upwind	0.25	0.0854
Lax-Wendroff	1	0
Lax-Wendroff	0.75	0.0442
Lax-Wendroff	0.5	0.0579
Lax-Wendroff	0.25	0.0751
Min-Dispersion	1	0
Min-Dispersion	0.75	0.0435
Min-Dispersion	0.5	0.0699
Min-Dispersion	0.25	0.1092

Again, we see that our results are the most accurate at a CFL = 1.

For part 4, something very interesting occurs. Each “scheme” looks the exact same. And when we look closer at the updated flux formula that we are using, it becomes very apparent. This new flux formula only depends on the CFL number. Previously, when we had a $q(v)$ term present, each scheme was handling the CFL number in a different manner. However, the $1 - v$ means that there are actually no different methods being tested out and each case is simply reliant on the CFL number. This can be seen in the figure below.

Comparing Nus for Schemes with B-Flux Formula and $aT = 0.5$



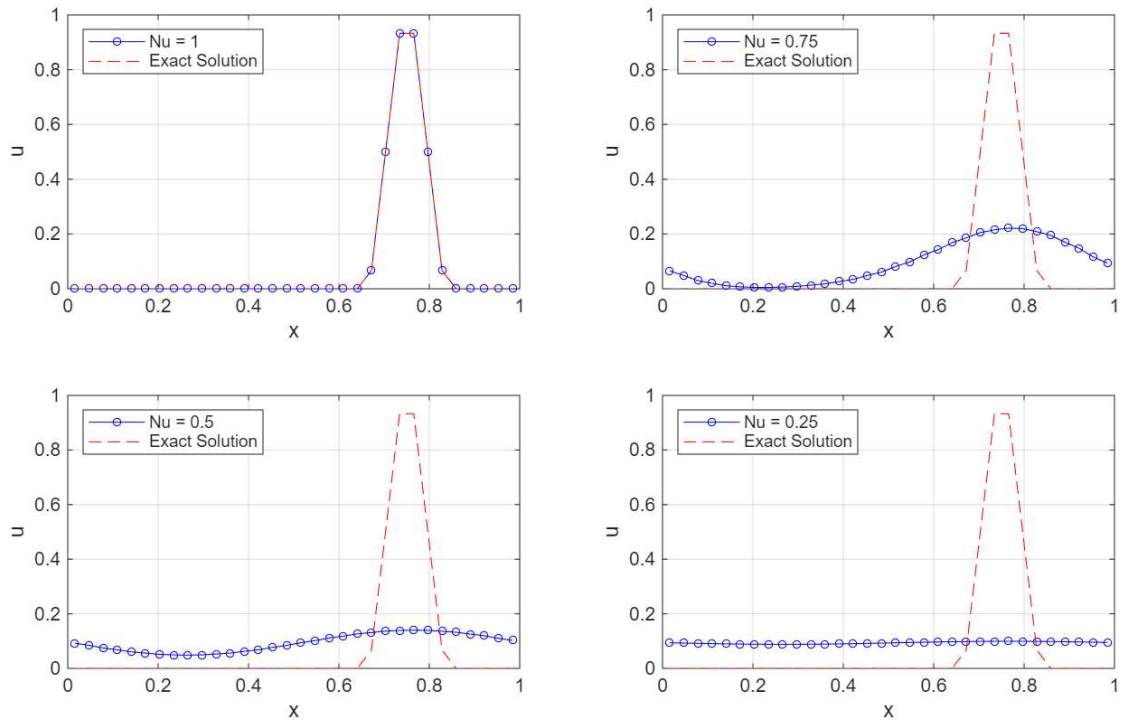
All the results look exactly the same!

Extra Credit –

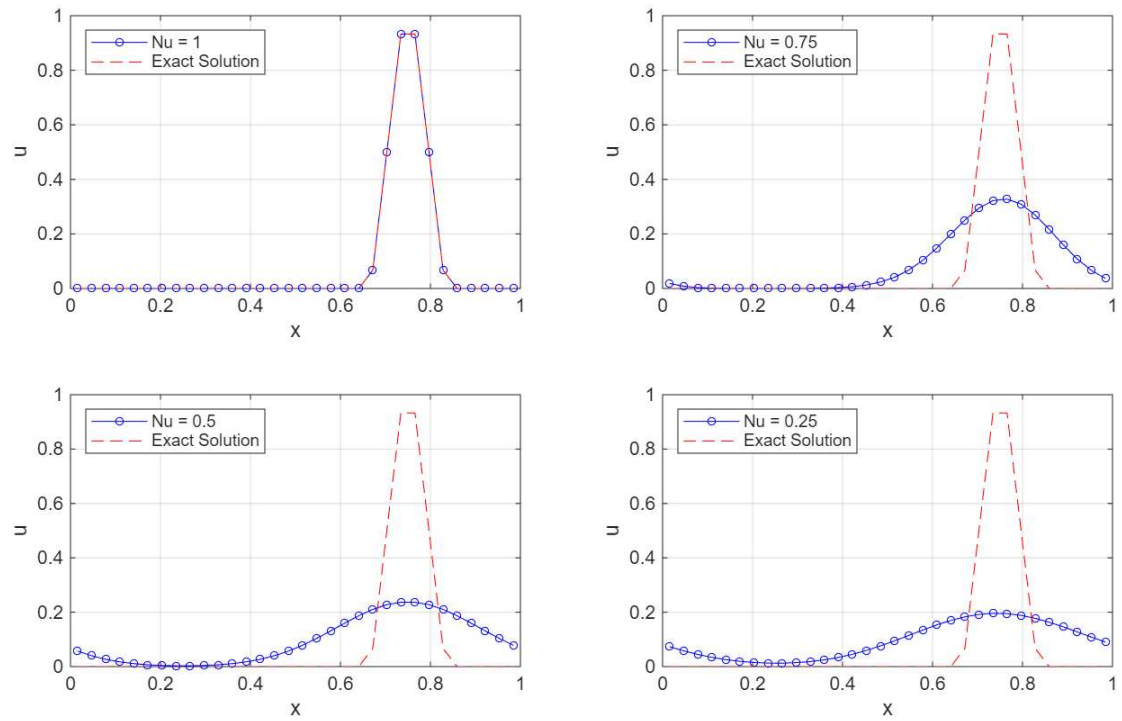
I predict that we will see the same trends in all of the different scenarios, where $CFL = 1$ is a very close or perfect match to the exact solutions, with the accuracy increasing as we increase our CFL number. However, I am curious to see what sort of behavior each scheme exhibits as it tries to match the wave packet with lower CFL number.

The same figures and data are generated from the different initial condition and are as follows-

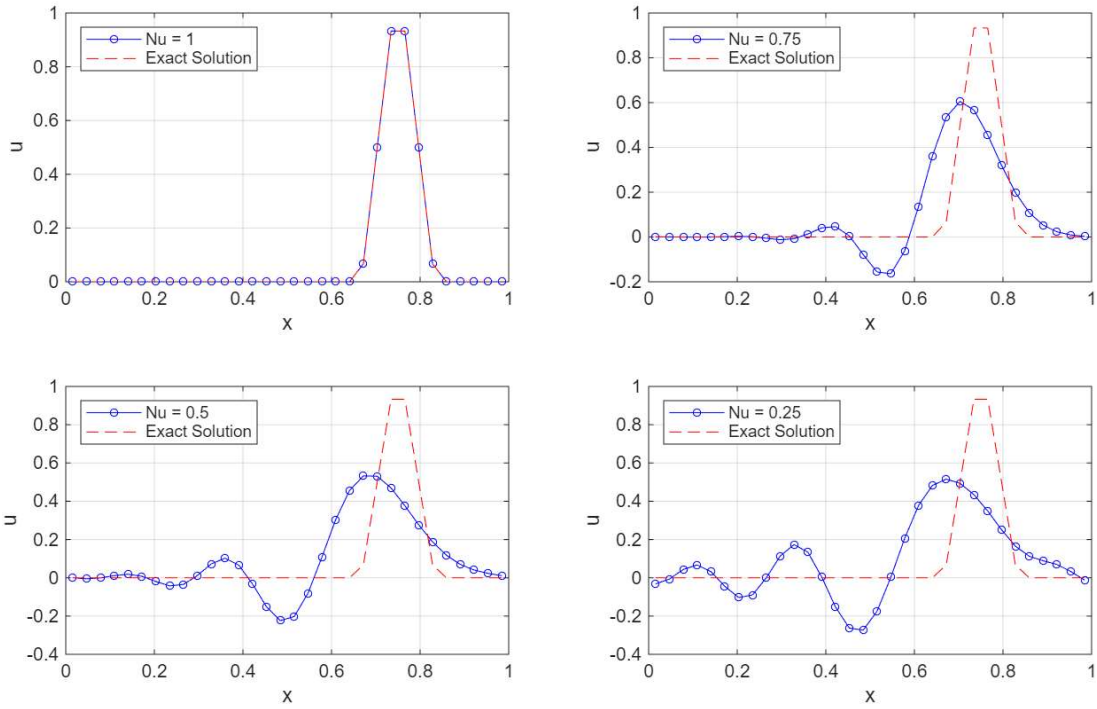
Comparing Nu Values for Lax-Friedrichs



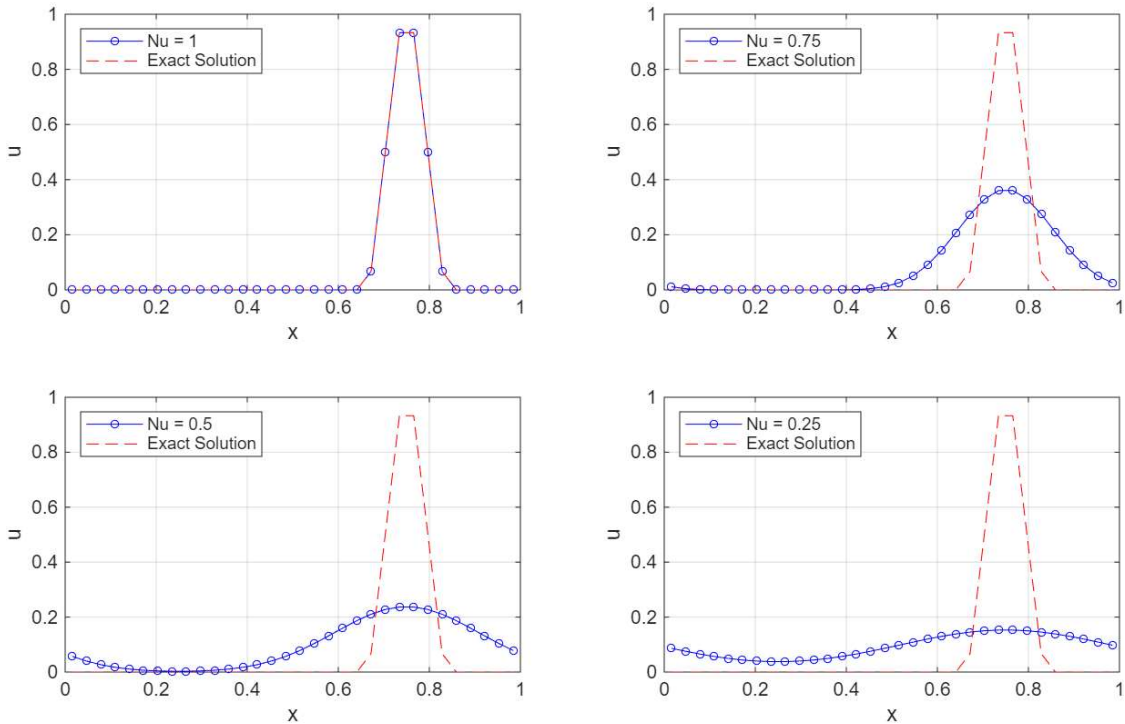
Comparing Nu Values for Upwind



Comparing Nu Values for Lax-Wendroff

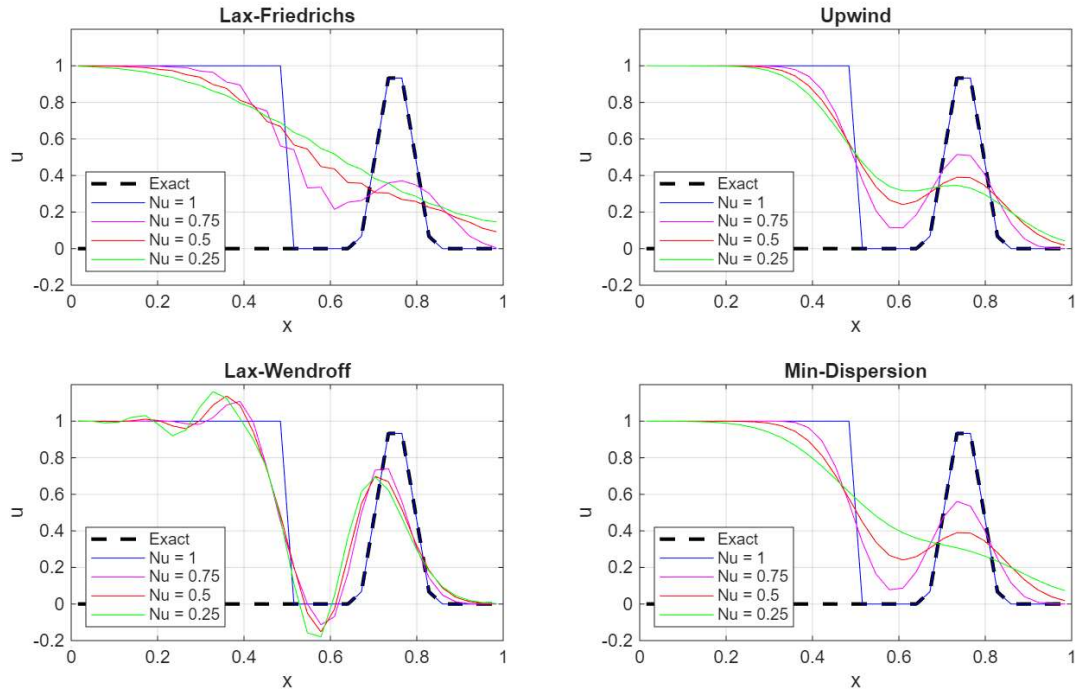


Comparing Nu Values for Min-Dispersion

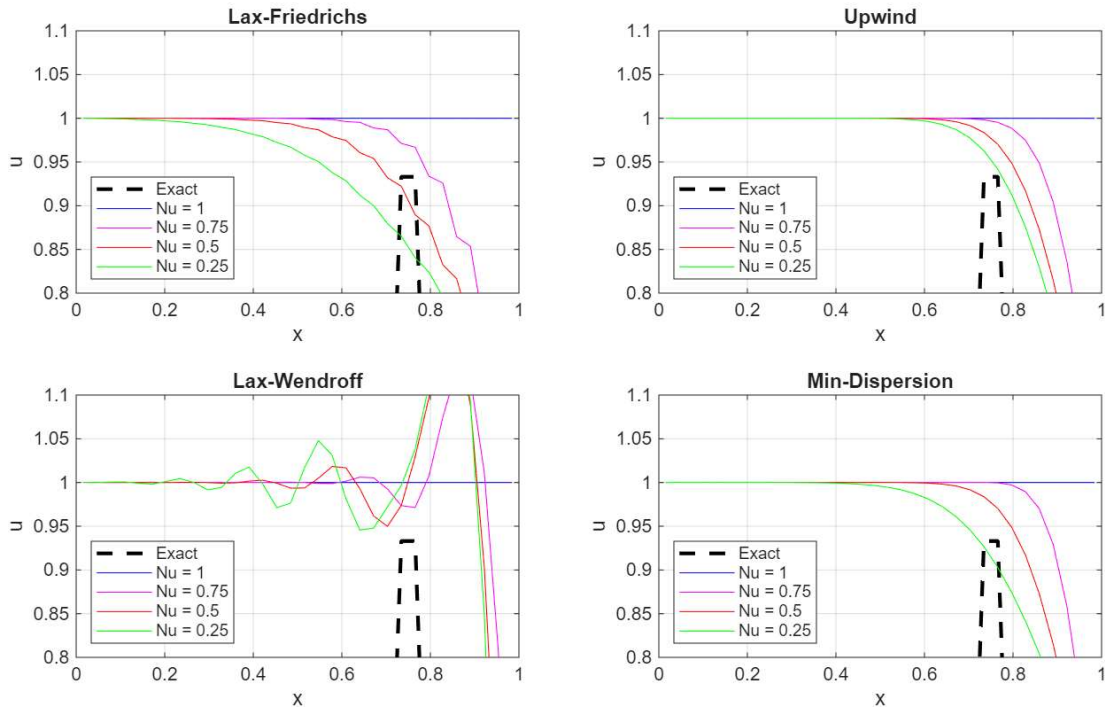


As predicted, a CFL number of 1 is the most accurate and matches exactly for all of these. However, there is much more insight gained into how each scheme appears to be operating and we can observe the progression in much more detail than the initial sinusoidal problem.

Comparing Nus for Schemes with $aT = 0.5$

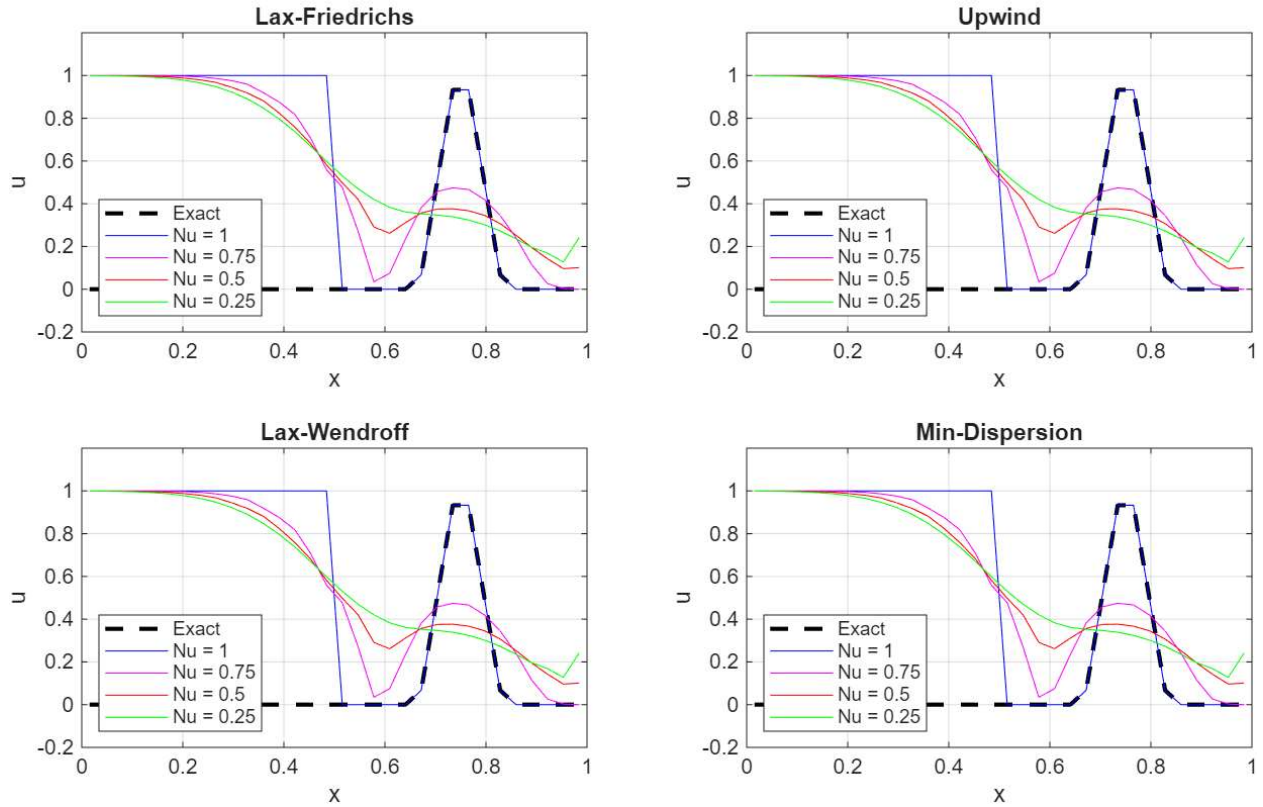


Comparing Nus for Schemes with $aT = 1$



Here we can see that the schemes all take much more time to adjust to the new wave packet from their initial starting conditions, however, again they manage to match the exact solution at least on the right side. I think the boundary conditions are playing a lot of interference on the left hand side as they still appear to be held at 1.

Comparing Nus for Schemes with B-Flux Formula and $aT = 0.5$



Again, for the final shockwave condition, all schemes collapse into the same behavior as there is no more dependency on what scheme is used, just the CFL number.

Appendix A – MATLAB Code

```
% Eleni Kambouris CFD Project 2 Code
clc
clear
close all

% Extra Credit Initial Conditions are commented and uncommented depending
% on the desired output of figures

%% Question 2
% Set up Schemes -
schemes = {
    @(nu) 1;                                % Lax-Friedrichs
    @(nu) abs(nu);                          % Upwind
    @(nu) nu.^2;                            % Lax-Wendroff
    @(nu) (1/3) + (2/3)*nu.^2              % Min dispersion
};

schemeNames = {'Lax-Friedrichs', 'Upwind', 'Lax-Wendroff', 'Min-Dispersion'};
% Set up Nu Options
nus = [1.0; 0.75; 0.50; 0.25];

% Set up time
a = 1;
T = 3/(2*a);

% Outer Loop for all 4 Schemes -
for s = 1:4
    scheme = schemeNames{s}; % for printing
    % select scheme and set up constants
    q_func = schemes{s};
    M = 32; dx = 1/M;
    x = (0:M-1)*dx + dx/2;

    % Extra Credit -
    % xi = 0.25; d = 3*dx;
    % u_packet = cos(pi*(x-xi)/d).^2 .* (abs(x-xi) <= d);

% Next Loop to Run Through Nu Options -
    for n = 1:4
        nu = nus(n);
        dt = (nu*dx)/a;
        t = 0:dt:T;
        N = ceil(T / dt); % Number of time steps
        T = N * dt;
        q = q_func(nu);

        % Compute exact results and set up boundary conditions and array
        u_exact = sin(2*pi*(x - a*T));
        u = sin(2*pi*x);

        % Extra Credit
        % u = u_packet;
        % cells_to_shift = round(1.5 / dx);
```

```

    % u_exact = circshift(u_packet, cells_to_shift);

    % Update the U values with time -

    for z = 1:N
        u = update_u(u, q, dx, dt, a);
    end

    % Compute amplitude and phase error
    [amp, phase] = fit_sine(x, u);
    amp_error = abs(amp) - 1;
    phase_error = -360 * phase;

    % L1 error
    L1 = mean(abs(u - u_exact));

    fprintf('%s, nu=%.2f: Amp err=%.4f, Phase err=%.2f deg, L1=%.4f\n', ...
            scheme, nu, amp_error, phase_error, L1);

    figure(s)
    subplot(2,2,n)
    plot(x,u,'bo-','MarkerSize',4)
    hold on
    grid on
    plot(x,u_exact,'r--')
    xlabel('x')
    ylabel('u')
    legend("Nu = " + num2str(nu), "Exact Solution","Location","NorthWest")
    sgtitle('Comparing Nu Values for ' + string(schemeNames(s)));
    hold off
end
end

%% Question 3
T = [0.5 1.0]; % leave a = 1
color = {'b','m','r','g'};

for t_bar = 1:2
    T_curr = T(t_bar);
    dist = 0.25 + T_curr; % exact position of shock
    u_exact = double(x <= dist);

    % Extra Credit -
    % xi = 0.25; d = 3*dx;
    % u_packet = cos(pi*(x-xi)/d).^2 .* (abs(x-xi) <= d);

    for s = 1:4
        scheme = schemeNames{s}; % for printing
        q_func = schemes{s};
        % plot the exact solution
        figure(4+t_bar)
        subplot(2,2,s)
        plot(x, u_exact, 'k--', 'LineWidth', 2);
        hold on
        for n = 1:4

```

```

nu = nus(n); dt = nu*dx/a; N = round(T_curr/(dt));
q = q_func(nu);
u = zeros(1,M); u(1:M/4) = 1;

% Extra Credit
% u = u_packet;
% cells_to_shift = round(1.5 / dx);
% u_exact = circshift(u_packet, cells_to_shift);

    for t = 1:N
        u = update_u_shock(u, q, dx, dt, a);
    end

    % plots of discrete schemes
    figure(4+t_bar)
    subplot(2,2,s)
    plot(x, u, color{n});
    title(string(scheme))
    xlabel('x')
    ylabel('u')
    grid on
    if t_bar == 1
        ylim([-0.2 1.2])
    elseif t_bar == 2
        ylim([0.8 1.1])
    end
    % L1 error
    L1 = mean(abs(u - u_exact));

    fprintf('%s, nu=%.2f:, L1=%.4f\n', ...
            scheme, nu, L1);

    % add a legend? without extra entries warning??
    legend('Exact', 'Nu = 1', 'Nu = 0.75', 'Nu = 0.5', 'Nu =
0.25', "Location", "Southwest")
    sgtitle('Comparing Nus for Schemes with aT = ' + string(T_curr));
    end

end

end

%% Question 4
T = 0.5; % leave a = 1
color = {'b', 'm', 'r', 'g'};

T_curr = T;
dist = 0.25 + T_curr; % exact position of shock
u_exact = double(x <= dist);

% Extra Credit -
% xi = 0.25; d = 3*dx;

```



```

% u_packet = cos(pi*(x-xi)/d).^2 .* (abs(x-xi) <= d);

for s = 1:4
    scheme = schemeNames{s}; % for printing
    q_func = schemes{s};
    % plot the exact solution
    figure(7)
    subplot(2,2,s)
    plot(x, u_exact, 'k--', 'LineWidth', 2);
    hold on
    for n = 1:4
        nu = nus(n); dt = nu*dx/a; N = round(T_curr/(dt));
        q = q_func(nu);
        u = zeros(1,M); u(1:M/4) = 1;

        % Extra Credit
        % u = u_packet;
        % cells_to_shift = round(1.5 / dx);
        % u_exact = circshift(u_packet, cells_to_shift);

        for t = 1:N
            u = update_u_shock_newFlux(u, nu, dx, dt);
        end

        % plots of discrete schemes
        figure(7)
        subplot(2,2,s)
        plot(x, u, color{n});
        title(string(scheme))
        xlabel('x')
        ylabel('u')
        grid on
        ylim([-0.2 1.2])

        % L1 error
        L1 = mean(abs(u - u_exact));

        fprintf('%s, nu=%.2f, L1=%.4f\n', ...
            scheme, nu, L1);

        % add a legend? without extra entries warning??
        legend('Exact', 'Nu = 1', 'Nu = 0.75', 'Nu = 0.5', 'Nu =
0.25', "Location", "Southwest")
        sgtitle('Comparing Nus for Schemes with B-Flux Formula and aT = ' +
string(T_curr));
    end

end

%% Helper Functions -

% Update Function
function u_new = update_u(u, q, dx, dt, a)
% Extend periodically

```

```

u_ext = [u(end), u, u(1)];

F = flux(u_ext, q, dx, dt, a);

% Update
u_new = u - (dt/dx) * (F(2:end) - F(1:end-1));
end

% Flux Function
function F = flux(u_ext, q, dx, dt, a)
M_ext = length(u_ext);
F = zeros(1, M_ext-1); % Fluxes at j+1/2

for j = 1:M_ext-1
    f_L = a * u_ext(j);
    f_R = a * u_ext(j+1);

    F(j) = 0.5*(f_L + f_R) - (q/2) * (dx/dt) * (u_ext(j+1) - u_ext(j));
end
end

% Amplitude and Phase -
function [A, phi] = fit_sine(x, u)
c = mean(u.*cos(2*pi*x));
s = mean(u.*sin(2*pi*x));
A = 2*sqrt(c^2 + s^2);
phi = atan2(s,c)/(2*pi);
end

% Shock Function
function u_new = update_u_shock(u, q, dx, dt, a)
u_ext = [1, u, u(end)];

F = flux(u_ext, q, dx, dt, a);

% Update
u_new = u - (dt/dx) * (F(2:end) - F(1:end-1));
end

% Shock Function - Part 4
function u_new = update_u_shock_newFlux(u, nu, dx, dt)
u_ext = [1, u, u(end)];

F = flux_4(u,u_ext,nu);

% Update
u_new = u - (dt/dx) * (F(2:end) - F(1:end-1));
end

% Part 4 Flux Function
function F = flux_4(u,u_ext, nu)

```

```

M = length(u);
delta = diff(u_ext); F = zeros(1,M+1);
for j = 1:M
    dxB = delta(j); dyB = delta(j+1);

    % Set up B Conditions -
    if sign(dxB)*sign(dyB) <= 0
        B = 0;
    else
        r = dxB/dyB;
        if abs(r) >= 0.5 && abs(r) <= 2.0
            B = max(abs(dxB), abs(dyB));
        else
            B = 2 * min(abs(dxB), abs(dyB));
        end
    end
    % Get Flux
    F(j) = u_ext(j) + (1-nu)/2 * B; % a = 1 so like don't really have to multiply
by it
end
end

```

Appendix B – Derivation

start w/ $\frac{\partial u}{\partial t} + \frac{\partial f}{\partial x} = 0$ w/ $f = uv$, $v = \frac{\alpha \Delta t}{\Delta x}$

and interface -

$$f_{j+1/2}^{n+1/2} = \frac{1}{2} (f(u_j^n) + f(u_{j+1}^n)) - \frac{q(v)}{2} \cdot \frac{\Delta x}{\Delta t} (u_{j+1}^n - u_j^n) \quad \text{w/ } v \equiv \frac{\alpha \Delta t}{\Delta x}$$

$$\hookrightarrow f_{j+1/2}^{n+1/2} = \frac{1}{2} (u_j^n + u_{j+1}^n) - \frac{q(v)}{2} \cdot \frac{\Delta x}{\Delta t} (u_{j+1}^n - u_j^n)$$

$$\frac{\partial f}{\partial x} (f_{j+1/2}^{n+1/2} = \frac{q}{2} (u_j^n + u_{j+1}^n) - \frac{q(v)}{2} \cdot \frac{\Delta x}{\Delta t} (u_{j+1}^n - u_j^n))$$

$$\frac{\Delta t}{\Delta x} \cdot f_{j+1/2}^{n+1/2} = \frac{v}{2} (u_j^n + u_{j+1}^n) - \frac{q(v)}{2} (u_{j+1}^n - u_j^n)$$

And $u_j^{n+1} = u_j^n - \frac{\Delta t}{\Delta x} (f_{j+1/2}^{n+1/2} - f_{j-1/2}^{n+1/2})$

$$\begin{aligned} \rightarrow u_j^{n+1} &= u_j^n - \left[\left(\frac{v}{2} u_j^n \right) + \left(\frac{v}{2} u_{j+1}^n \right) - \left(\frac{q(v)}{2} u_{j+1}^n \right) + \left(\frac{q(v)}{2} u_j^n \right) - \left(\frac{v}{2} u_{j-1}^n \right) + \left(\frac{v}{2} u_j^n \right) - \left(\frac{q(v)}{2} u_j^n \right) + \left(\frac{q(v)}{2} u_{j-1}^n \right) \right] \\ &= u_j^n \left(1 - \frac{v}{2} - \frac{q(v)}{2} + \frac{v}{2} - \frac{q(v)}{2} \right) \\ &\quad + u_{j+1}^n \left(\frac{v}{2} - \frac{q(v)}{2} \right) \\ &\quad + u_{j-1}^n \left(-\frac{v}{2} + \frac{q(v)}{2} \right) \end{aligned}$$

$$\begin{aligned} \rightarrow c_0 &= 1 - q(v) \\ c_{+1} &= \frac{v - q(v)}{2} \\ c_{-1} &= \frac{q(v) - v}{2} \end{aligned}$$

$$\rightarrow u_j^{n+1} = \frac{v - q(v)}{2} \cdot u_{j+1}^n + (1 - q(v)) u_j^n + \frac{q(v) - v}{2} \cdot u_{j-1}^n$$

for Von Neumann Analysis $\rightarrow z^{n+1} e^{i\theta} = c_{-1} z^n e^{i\theta} e^{-i\theta} + c_0 z^n e^{i\theta} + c_{+1} z^n e^{i\theta} e^{i\theta}$

$$z = c_{-1} e^{-i\theta} + c_0 + c_{+1} e^{i\theta}$$

$$\star z = 1 - q + q \cos \theta - i v \sin \theta$$